GRADE 1 · INTRO DEBUGGING

The Missing Block

Add the block that's missing

Name: .... Date: ....

★ **I can fix a program by adding a missing block.**

Not far enough!

Sometimes a program is MISSING a block, so Bit falls short and never reaches the target. Run the code, see how far short it is, and add the missing block in the empty space.

Bit's number path

0	1	2	3	4	5

Program 1 — target is 4

start at 1

forward 2

[ ]

Write the missing block in the empty space.

1 Run it — too short

Run Program 1 from 1 (forward 2). What number does Bit land on? How many more does it need to reach 4?

____

____

2 Add the missing block

Write the block that gets Bit from where it stopped to 4. Run the fixed code — what number now?

Program 2 — target is 1

start at 4

back 1

Write the missing block in the empty space.

3 You try — add to fix

Run Program 2 from 4 (back 1). It lands on 3, not 1. Write the missing block so Bit reaches 1. What block did you add, and where does Bit land?

Big idea

If a program falls short, it may be missing a block. Add the step that finishes the job so Bit lands right on the target.

TEACHER GUIDE · GRADE 1 · INTRO DEBUGGING

The Missing Block

Quick facilitation notes & answer key

What this worksheet teaches

Students fix a bug where the program falls SHORT of the target because a block is missing; they add the missing step and run again. Adding a step to fix the result is a kind of debugging. No coding experience needed.

Before you start (no computer needed)

No computer — paper and a pencil. You read each prompt aloud. Optional: tape a number line 0 to 5 on the floor and mark the target, so students can walk the buggy program and feel Bit miss.

How to lead it (about 20 minutes)

1. Show them (you do). Run a program that stops short of the target — it is missing a step.
2. Do one together. Exercise 1 — Program 1 stops on 3, one short of 4.
3. Let them try. Students write the missing block to reach the target (Ex 2), then fix Program 2 (Ex 3).

Answer key (these are the verified correct answers)

Exercise 1 — Program 1: 1, forward 2 → 3. Bit is on 3, one short of 4.

Exercise 2 — add "forward 1": 1 → 3 → 4. Bit reaches the target, 4.

Exercise 3 — Program 2: 4, back 1 → 3. Add "back 2": 3 → 1. Bit reaches the target, 1. (Two "back 1" blocks also work.)

If students get stuck — and ways to adjust

Watch for: changing a block instead of adding one. Here the fix is to put a step in.

Make it easier: count how far Bit still needs to go, then write a block for that distance.

Make it harder: ask for two different missing blocks that both reach the target.

Ontario curriculum link (official wording)

C3.1 — solve problems and create computational representations of mathematical situations by writing and executing code, including code that involves sequential events.

In plain terms: students follow (or write) step-by-step instructions, in order, to get a result.

C3.2 — read and alter existing code, including code that involves sequential events, and describe how changes to the code affect the outcomes.

In plain terms: students read a program, change a block to fix it, and explain what the change did.

You'll know it worked when

The child adds the missing block so Bit reaches the target.